

1. Базовые элементы процессора и аппаратный уровень

Функциональные блоки процессора

- **Дешифратор команд (ДС):** расшифровывает код операции (КОП) текущей инструкции.
- **Операционные устройства (ОУ / АЛУ):** выполняют арифметические, логические операции и сдвиги.
- **Устройство управления и синхронизации (УУ / УУиС):** координирует работу всех узлов процессора и вырабатывает управляющие сигналы.
- **Регистры:** внутренняя сверхбыстрая память для операндов, адресов и служебных данных.
- **Транслятор адресов операндов:** вычисляет исполнительные адреса в памяти.

Внутренние и внешние шины: связывают внутренние регистры между собой и обеспечивают обмен с системной шиной.

Внешний интерфейс микропроцессора (УГО)

- **Шина адреса (А):** однонаправленные выводы для указания адреса в памяти или периферии.
- **Шина данных (D):** двунаправленные выводы для приема и передачи данных.
- **Управляющие сигналы:** чтение/запись памяти (RD/WR, ЧтОП/ЗпОП), ввод/вывод на периферию (IN/OUT), синхронизация (CLC), питание (+5V, GND).

Служебные сигналы: SET (включение/пуск, инициализирует счетчик команд сигналом \$y_0\$), RESET (сброс).

Историческая справка: первый микропроцессор Intel 4004 (1971 г.) стоил 200\$, содержал 2300 транзисторов на кристалле размером с ноготь, работал на частоте 108 кГц, потреблял несколько десятков ватт и был сопоставим по вычислительной мощности с ламповой ЭВМ ENIAC.

2. Операционные устройства и схемотехника АЛУ

Структурный базис ОУ

- **Регистры:** линейки триггеров со стробируемыми входами. Классифицируются на явно программно-видимые (РОН), косвенно-видимые («теневые» дескрипторы), внутренние специальные (регистры адреса и данных памяти) и внутренние для промежуточных результатов.
- **Шины:** физические проводники для передачи машинных слов.

Комбинационные схемы: логические цепочки без элементов памяти, вычисляющие функции по сигналам от УУ.

Принцип работы АЛУ

- Представляет собой комбинационную схему, принимающую два операнда и формирующую результат на выходе.
- **Состав схемы 1-битного АЛУ:** логический блок (операции AND, OR, инверсия), полный сумматор (сложение с учетом переноса) и декодер управляющих линий.

Многоразрядные АЛУ: строятся последовательным каскадированием 1-битных модулей с передачей сигнала переноса от младших разрядов к старшим.

Временные диаграммы микроопераций

- **Пересылка (mov R1, R2):** открытие вентилей по стробу \$T_1\$.
- **Циклический сдвиг (rol R0):** передача содержимого R0 в буферный регистр BR со смещением шины на 1 бит (строб \$T_2\$), затем возврат в R0 (строб \$T_3\$).

Сложение (add R0, R1): на тактах \$t_1\$-\$t_2\$ (стробы \$T_4\$, \$T_5\$) операнды копируются во входные регистры АЛУ (RA и RB). На тактах \$t_2\$-\$t_5\$ (строб \$T_6\$) АЛУ выполняет операцию. На тактах \$t_4\$-\$t_5\$ (строб \$T_7\$) результат из выходного регистра RC записывается обратно в регистр-приемник.

Магистральные структуры ОПУ

- **Одномагистральная:** одна общая шина, требуются входные буферные регистры (БР1, БР2) перед операционным блоком (ОПБ).
- **Двухмагистральная:** две шины (например, A и B), один операнд подается напрямую, второй — через буферный регистр.

Трехмагистральная: две шины источника (A и B) через мультиплексоры подают данные на входы ОПБ, третья шина (C) через демультиплексор возвращает результат в узел РОН за один такт.

3. Устройство управления (УУ) и микропрограммирование

Базовые понятия

- **Микрооперация:** неделимое элементарное действие, выполняемое за один такт синхронизации.

- **Микрокоманда:** набор микроопераций, выполняемых аппаратно параллельно в рамках одного такта.

Микропрограмма: последовательность микрокоманд, реализующая алгоритм выполнения машинной команды.

Структура УУ

- *Управляющая часть:* регистр команд (РК), микропрограммный автомат (МПА), дешифратор команд, узел прерываний и приоритетов.

Адресная часть: счетчик команд (СК / СЧАК), регистр адреса, операционный узел адреса.

Микропрограммный автомат с жесткой логикой

- Управляющие сигналы вырабатываются комбинационными схемами на основе кода КОП, тактовых импульсов (счетчик тактов сбрасывается в T_1 по завершении цикла) и флагов.

Обеспечивает максимальное быстродействие, но аппаратно неизменяем.

Микропрограммный автомат с программируемой логикой

- Концепция предложена Морисом Уилксом (1951 г.). Команда ЭВМ интерпретируется микропрограммой, считываемой из специальной управляющей памяти (памяти микропрограмм).
- *Форматы микрокоманд:*
 - GATE: битовая маска, где каждый бит управляет открытием конкретного вентилля шины.

TEST: микрокоманда условного перехода к следующему адресу микрокода по состоянию бита флага.

- *Оптимизация памяти микрокода:*
 - Одноуровневая: $1000 \text{ адресов} \times 100 \text{ бит} = 100\,000 \text{ бит}$.

Двухуровневая (с нанопамятью): микропамять хранит 6-битные указатели ($1000 \times 6 = 6000 \text{ бит}$), нанопамять содержит 64 уникальных 100-битных слова ($64 \times 100 = 6400 \text{ бит}$), итого $12\,400 \text{ бит}$ (экономия памяти более чем в 8 раз).

4. Классификация архитектур системы команд (АСК)

Архитектура	Размещение операндов	Формат команд	Достоинства	Недостатки
Стековая	Вершина безадресного аппаратного/программного стека (LIFO).	Безадресный, операции <code>push/pop</code> . Используется обратная польская запись (ОПЗ).	Минимальные издержки на адресацию, предельно короткий код.	Сложность произвольного доступа к операндам, задержки стека.
Аккумуляторная	Один операнд и результат — в аккумуляторе; второй — в ОП.	Одноадресный (<code>load x, store x</code>).	Короткие команды, простая логика декодирования.	Высокая нагрузка на шину памяти из-за частых обращений к ОП.
Регистровая	В памяти или в массиве регистров общего назначения (РОН).	Двух- и трехадресные: «регистр–регистр», «регистр–память», «память–память».	Быстрый доступ к РОН (работают как явный кэш), компактный код.	Длинные машинные команды по сравнению с аккумуляторными.
С выделенным доступом (Load/Store)	Только в регистрах РОН. В память обращаются только <code>load</code> и <code>store</code> .	Фиксированная длина (обычно 32 бита), трехадресный формат.	Базис RISC-систем (SPARC, MIPS, Alpha), простота конвейеризации.	Увеличенный размер кода программы.

Обратная польская нотация (ОПЗ, Я. Лукашевич): постфиксная запись без скобок, где оператор следует за операндами: выражение $a = a + b + a \times c$ преобразуется в вид $a = a \ b \ + \ a \ c \ \times \ + \$$.

****Обратная польская нотация (ОПЗ, Я. Лукашевич):**** постфиксная запись без скобок, где оператор следует за операндами: выражение $a = a + b + a \times c$ преобразуется в вид $a = a \ b \ + \ a \ c \ \times \ + \$$.

Конфигурируемые процессоры: архитектуры (например, Tensilica Xtensa), позволяющие на этапе САПР (среда Xenergy) добавлять пользовательские регистровые файлы и инструкции, оптимизируя ядро под энергопотребление и конкретные задачи.

5. Архитектура процессора Intel 8086

- **Классификация:** 16-битный процессор класса CISC, включающий 538 инструкций (210 целочисленных, 85 с плавающей точкой, расширения MMX/XMM). Длина команд составляет от 1 до 15 байт.
- **Аппаратные узлы:**

- **УО (Устройство обработки):** арифметический сумматор, внутренние регистры, регистр флагов FL.

УСШ (Устройство сопряжения с шиной): 6-байтная очередь предвыборки команд, сумматор адреса, сегментные регистры. Внутренняя шина данных — 16 бит, внешняя шина адреса — 20 бит (адресует до 1 МБ ОП).

- **Регистровая модель:**

- *Данные и адреса:* AX (аккумулятор), BX (база), CX (счетчик), DX (данные), SI (индекс источника), DI (индекс приемника), SP (указатель стека), BP (указатель базы), IP (указатель команд).

Сегментные регистры: CS (код), DS (данные), SS (стек), ES (дополнительный сегмент).

- **Сегментная организация памяти:**

- Размер любого сегмента — до 64 КБ (2^{16} байт).

Формула вычисления 20-битного физического адреса:

$$\text{Физический адрес} = (\text{Сегментный регистр} \times 16) + \text{Смещение (Offset)}$$

- *(Сдвиг сегментного регистра на 4 бита влево с последующим сложением со смещением).*

Модели памяти: Tiny, Small, Medium, Compact, Large, Huge.

- **Регистр флагов (16 бит):**

- *Арифметические флаги:* CF (перенос), PF (четность), AF (полуперенос/вспомогательный перенос), ZF (нуль), SF (знак), OF (переполнение).

Системные/управляющие флаги: TF (пошаговая отладка), IF (разрешение маскируемых прерываний), DF (направление обработки строк).

6. Представление данных и формат команд

Представление данных

- **Числа:** знаковые и беззнаковые форматы с фиксированной точкой (старший бит 0 — плюс, 1 — минус) и с плавающей точкой. Пример: $01101101_2 = 1 + 4 + 8 + 32 + 64 = +109_{10}$.
- **Символы:** представляются двоичными кодами (ASCII и др.).

Логические векторы: битовые цепочки над беззнаковыми типами.

Формат двухоперандной команды

Содержит три ключевых поля: [КОП | Адрес 1-го операнда (A1) | Адрес 2-го операнда (A2)]. Результат операции, как правило, записывается по адресу первого операнда \$A_1\$.

Пример системы команд (микроконтроллер KM1816BE51)

- **Сложение:** ADD A, Rn, ADD A, ad, ADD A, #d, с переносом ADDC, десятичная коррекция DA A.
- **Вычитание с заемом:** SUBB A, Rn / ad / @Ri / #d.
- **Инкремент/декремент:** INC, DEC (аккумулятора, регистров, прямого адреса, указателя DPTR).

Умножение и деление: MUL AB (результат в BA), DIV AB (частное в A, остаток в B).

7. Полный рабочий цикл процессора

Рабочий цикл выполнения машинной команды включает 5 строго синхронизированных этапов:

None

```
graph LR
    1["[1. Выборка команды]"] --> 2["[2. Выборка операндов]"]
    2 --> 3["[3. Выполнение в АЛУ]"]
    3 --> 4["[4. Запись результата]"]
    4 --> 5["[5. Продвижение СЧАК]"]
    5 --> 1
```

...

Этап 1. Выборка команды из ОП

- \$y_1\$: ПортA := СЧАК — процессор выставляет адрес текущей команды на шину адреса (в регистр РАОП).
- \$y_2\$: ЧтОП — на шину управления подается строб чтения.

\$y_3\$: РК := ПортД — считанное из PCОП машинное слово через шину данных поступает в регистр команд (РК).

Этап 2. Выборка операндов

- **Выборка операнда 1:**

- \$y_4\$: ПортА := А1 — адрес операнда \$A_1\$ из РК выставляется на шину адреса.
- \$y_2\$: ЧтОП — сигнал чтения памяти.
- \$y_5\$: P1 := ПортД — значение помещается в первый регистр (аккумулятор) АЛУ.

- **Выборка операнда 2:**

- \$y_6\$: ПортА := А2 — адрес операнда \$A_2\$ выставляется на шину адреса.
- \$y_2\$: ЧтОП — сигнал чтения памяти.

\$y_7\$: P2 := ПортД — значение загружается во второй регистр АЛУ.

Этап 3. Выполнение операции в АЛУ

- КОП из РК дешифруется на DC, формируя осведомительный сигнал \$x_i = 1\$. УУиС в ответ генерирует сигнал \$y_i\{АЛУ\} = 1\$, запуская микропрограмму операции.

АЛУ выставляет сигнал занятости \$Z_{\{АЛУ\}} = 1\$ (линия ШАЛУ(8)). Процессор ожидает сброса \$Z_{\{АЛУ\}} = 0\$, сигнализирующего об окончании вычислений и фиксации результата в аккумуляторе P1.

* АЛУ выставляет сигнал занятости \$Z_{\{АЛУ\}} = 1\$ (линия ШАЛУ(8)). Процессор ожидает сброса \$Z_{\{АЛУ\}} = 0\$, сигнализирующего об окончании вычислений и фиксации результата в аккумуляторе P1.

Этап 4. Запись результата в память и фиксация флагов

Этап 4. Запись результата в память и фиксация флагов

- \$y_8\$: ПортД := P1 — результат переносится в выходной буфер данных.
- \$y_4\$: ПортА := А1 — на шину адреса подается адрес приемника \$A_1\$.
- \$y_9\$: ЗпОП — импульс записи фиксирует данные в ОП.

Параллельная фиксация флагов:

* *Параллельная фиксация флагов:*

- \$y_{11}\$: РПР(0:1) := Coder(ШАЛУ(0:3)) — унитарный код признаков результата шифратором преобразуется в 2-битный позиционный код регистра признака результата (РПР):
 - 00: результат равен 0 (\$P1 = 0\$).
 - 01: результат меньше 0 (\$P1 < 0\$).
 - 10: результат больше 0 (\$P1 > 0\$).

11: переполнение с фиксированной точкой (ППФТ).

y_{12} : $RF(0:4) := ШАЛУ(3:7)$ — исключительные ситуации (переполнение порядка, исчезновение порядка, потеря значимости, деление на ноль) фиксируются в регистре флагов (РФ).

Этап 5. Продвижение счетчика адреса команд

14. y_{10} : $СчАК := СчАК + L$ — значение счетчика увеличивается на длину выполненной команды L в байтах для перехода к следующей команде.

14. y_{10} : $\text{`СчАК} := СчАК + L$ — значение счетчика увеличивается на длину выполненной команды L в байтах для перехода к следующей команде.

8. Команды передачи управления (ветвление по маске)

- **Назначение:** изменение линейного порядка выполнения программы для организации циклов и ветвлений.
- **Формат команды условного перехода по маске:**
 - $[КОП: РК(0:7) \mid \text{Маска } M: РК(8:11) \mid \text{Адрес перехода } A: РК(12:n)]$.
- **Логика анализа маски:**
 - Если $M = 0000$ или $A = 0$ — переход не выполняется (эквивалентно NOP).
 - Если $M = 1111$ — безусловный переход (эквивалентно JMP).

Соответствие битов маски: бит $РК(8)$ проверяет условие $=0$, $РК(9)$ проверяет <0 , $РК(10)$ проверяет >0 , $РК(11)$ проверяет переполнение (ППФТ).

Аппаратная реализация: логическая схема проверяет совпадение значений $РПР(0:1)$ (через дешифратор DC) с соответствующим битом маски через логические элементы «И», сводя результат на элемент «ИЛИ» (сигнал x_4). Если условие выполняется, вырабатывается сигнал y_1 : $СчАК := РК(12:n)$, иначе управление передается по ранее продвинутому адресу.

*****Аппаратная реализация:**** логическая схема проверяет совпадение значений $РПР(0:1)$ (через дешифратор DC) с соответствующим битом маски через логические элементы «И», сводя результат на элемент «ИЛИ» (сигнал x_4). Если условие выполняется, вырабатывается сигнал y_1 : $\text{`СчАК} := РК(12:n)$, иначе управление передается по ранее продвинутому адресу.

Разрядность адреса: определяется объемом оперативной памяти:

$$N_A = \lceil \log_2 E_{OP} \rceil$$

Такую разрядность имеют счетчик СчАК и адресное поле A команды.

9. Подсистема прерываний процессора

Классификация прерываний

- **Внутренние (исключения / аварии):** деление на ноль, переполнение порядка, нарушение адресации. Не могут быть замаскированы.
- **Внешние:** запросы от внешних контроллеров (клавиатура, диск). Могут маскироваться через Регистр масок (PM).

Прерывания по таймеру: квантование процессорного времени ($\tau = 1/30 \text{ с}$) для обеспечения иллюзии параллельной работы программ в мультипрограммном режиме.

Аппаратная база прерываний

*****Аппаратная база прерываний:*****

- **Регистр флагов (РФ):** каждый i -й бит соответствует возникновению i -й причины прерывания.
- **Регистр масок (РМ):** единичный бит запрещает обработку соответствующего прерывания.
- **Указатель стека (УС / SP):** хранит адрес вершины аппаратного/программного стека (LIFO).

Таблица векторов прерываний (ТВП): первые n ячеек ОП, где по i -му адресу жестко записана команда безусловного перехода `JMP Ai` на подпрограмму обработки i -го прерывания.

None

Векторное прерывание:

[Шаг 1] ОП[УС] := СК; УС := УС + 1	(Сохранение адреса возврата в стек)
[Шаг 2] СК := i	(Выбор вектора из ТВП)
[Шаг 3] Выполнение JMP Ai	(Переход на обработчик)
[Шаг 4] Исполнение тела обработчика	
[Шаг 5] Выполнение RETI:	(Извлечение адреса возврата из стека)
УС := УС - 1; СК := ОП[УС]	

Сохранение контекста: базовая схема сохраняет в стеке только адрес возврата СК. За сохранение используемых регистров РОН и РПР отвечает программист

обработчика, либо процессор автоматически сохраняет в стеке полное **Слово состояния программы (ССП / PSW)**.